

Ex. No. 1A Date: 08/07/2026	CREATING AND MANAGING TABLES	URK24CS1021
Aim To execute DDL commands and get the desired output.		
Description DDL refers to "Data Definition Language", a subset of SQL statements that change the structure of the database schema in some way, typically by creating, deleting, or modifying schema objects such as databases, tables, and views. Most DDL statements start with the keywords CREATE, DROP, or ALTER.		
Questions 1. Create User, Event, Venue, and ticket tables based on the given schema.		
<pre> CREATE TABLE URK24CS1021_users (userid numeric(10) PRIMARY KEY, name varchar(255), email varchar(255), password varchar(255), phone varchar(20)); CREATE TABLE URK24CS1021_venue (venueid numeric(10) PRIMARY KEY, name varchar(255), address varchar(255), city varchar(255), state varchar(255), country varchar(255)); CREATE TABLE URK24CS1021_event (eventid numeric(10) PRIMARY KEY, name varchar(255), eventdate date, eventtime timestamp, venueid numeric(10), description varchar(500)); CREATE TABLE URK24CS1021_ticket (ticketid numeric(10) PRIMARY KEY, eventid numeric(10) REFERENCES URK24CS1021_event(eventid), userid numeric(10), seatnumber varchar(20), price numeric(10,2), status varchar(50)); </pre>		

psql — labdb

```
labdb=# CREATE TABLE URK24CS1021_users (  
labdb(#      userid numeric(10) PRIMARY KEY,  
labdb(#      name varchar(255),  
labdb(#      email varchar(255),  
labdb(#      password varchar(255),  
labdb(#      phone varchar(20)  
labdb(# );  
CREATE TABLE  
  
labdb=# CREATE TABLE URK24CS1021_venue (  
labdb(#      venueid numeric(10) PRIMARY KEY,  
labdb(#      name varchar(255),  
labdb(#      address varchar(255),  
labdb(#      city varchar(255),  
labdb(#      state varchar(255),  
labdb(#      country varchar(255)  
labdb(# );  
CREATE TABLE  
  
labdb=# CREATE TABLE URK24CS1021_event (  
labdb(#      eventid numeric(10) PRIMARY KEY,  
labdb(#      name varchar(255),  
labdb(#      eventdate date,  
labdb(#      eventtime timestamp,  
labdb(#      venueid numeric(10),  
labdb(#      description varchar(500)  
labdb(# );  
CREATE TABLE  
  
labdb=# CREATE TABLE URK24CS1021_ticket (  
labdb(#      ticketid numeric(10) PRIMARY KEY,  
labdb(#      eventid numeric(10) REFERENCES URK24CS1021_event(eventid),  
labdb(#      userid numeric(10),  
labdb(#      seatnumber varchar(20),  
labdb(#      price numeric(10,2),  
labdb(#      status varchar(50)  
labdb(# );  
CREATE TABLE  
  
labdb=#
```

2. Describe the tables.

```
\d URK24CS1021_users  
\d URK24CS1021_event  
\d URK24CS1021_venue  
\d URK24CS1021_ticket
```

```
psql -- labdb

labdb=# \d URK24CS1021_users
               Table "public.urk24cs1021_users"
  Column |          Type          | Collation | Nullable | Default
-----+-----+-----+-----+-----
userid  | numeric(10,0)           |           | not null |
name    | character varying(255)  |           |          |
email   | character varying(255)  |           |          |
password| character varying(255)  |           |          |
phone   | character varying(20)   |           |          |
Indexes:
    "urk24cs1021_users_pkey" PRIMARY KEY, btree (userid)

labdb=# \d URK24CS1021_event
               Table "public.urk24cs1021_event"
  Column |          Type          | Collation | Nullable | Default
-----+-----+-----+-----+-----
eventid | numeric(10,0)           |           | not null |
name    | character varying(255)  |           |          |
eventdate | date                   |           |          |
eventtime | timestamp without time zone |           |          |
venueid  | numeric(10,0)           |           |          |
description | character varying(500) |           |          |
Indexes:
    "urk24cs1021_event_pkey" PRIMARY KEY, btree (eventid)
Referenced by:
    TABLE "urk24cs1021_ticket" CONSTRAINT "urk24cs1021_ticket_eventid_fkey" FOREIGN KEY (eventid) REFERENCES urk24cs1021_event(eventid)

labdb=# \d URK24CS1021_venue
               Table "public.urk24cs1021_venue"
  Column |          Type          | Collation | Nullable | Default
-----+-----+-----+-----+-----
venueid | numeric(10,0)           |           | not null |
name    | character varying(255)  |           |          |
address | character varying(255)  |           |          |
city    | character varying(255)  |           |          |
state   | character varying(255)  |           |          |
country | character varying(255)  |           |          |
Indexes:
    "urk24cs1021_venue_pkey" PRIMARY KEY, btree (venueid)

labdb=# \d URK24CS1021_ticket
               Table "public.urk24cs1021_ticket"
  Column |          Type          | Collation | Nullable | Default
-----+-----+-----+-----+-----
ticketid | numeric(10,0)           |           | not null |
eventid  | numeric(10,0)           |           |          |
userid   | numeric(10,0)           |           |          |
seatnumber | character varying(20)   |           |          |
price    | numeric(10,2)           |           |          |
status   | character varying(50)   |           |          |
Indexes:
    "urk24cs1021_ticket_pkey" PRIMARY KEY, btree (ticketid)
Foreign-key constraints:
    "urk24cs1021_ticket_eventid_fkey" FOREIGN KEY (eventid) REFERENCES urk24cs1021_event(eventid)

labdb=#
```

3. Alter the User table to add a new column Age.

```
ALTER TABLE URK24CS1021_users ADD COLUMN age numeric(3);
```

```
psql -- labdb

labdb=# ALTER TABLE URK24CS1021_users ADD COLUMN age numeric(3);
ALTER TABLE

labdb=#
```

4. Drop the newly added column Age.

```
ALTER TABLE URK24CS1021_users DROP COLUMN age;
```

psql — labdb

```
labdb=# ALTER TABLE URK24CS1021_users DROP COLUMN age;  
ALTER TABLE
```

labdb=#

5. Rename the Venue table to Location.

```
ALTER TABLE URK24CS1021_venue RENAME TO URK24CS1021_location;
```

psql — labdb

```
labdb=# ALTER TABLE URK24CS1021_venue RENAME TO URK24CS1021_location;  
ALTER TABLE
```

labdb=#

6. Modify the size of the Event table's Description column to 1000.

```
ALTER TABLE URK24CS1021_event ALTER COLUMN description TYPE varchar(1000);
```

psql — labdb

```
labdb=# ALTER TABLE URK24CS1021_event ALTER COLUMN description TYPE varchar(1000);  
ALTER TABLE
```

labdb=#

7. Drop the SeatNumber column from the Ticket table.

```
ALTER TABLE URK24CS1021_ticket DROP COLUMN seatnumber;
```

psql — labdb

```
labdb=# ALTER TABLE URK24CS1021_ticket DROP COLUMN seatnumber;  
ALTER TABLE
```

labdb=#

8. Add a unique constraint on the Email column in the User table.

```
ALTER TABLE URK24CS1021_users ADD CONSTRAINT users_email_uq UNIQUE (email);
```

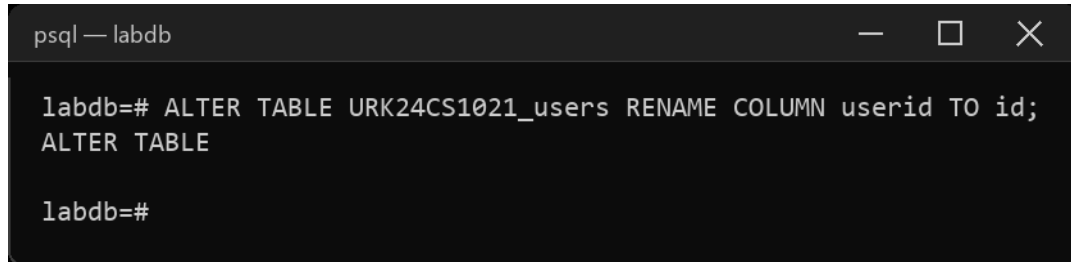
psql — labdb

```
labdb=# ALTER TABLE URK24CS1021_users ADD CONSTRAINT users_email_uq UNIQUE (email);  
ALTER TABLE
```

labdb=#

9. Rename the UserID column in the User table to ID.

```
ALTER TABLE URK24CS1021_users RENAME COLUMN userid TO id;
```

A terminal window titled 'psql — labdb' with standard window controls. It shows the execution of the SQL command 'ALTER TABLE URK24CS1021_users RENAME COLUMN userid TO id;'. The prompt 'labdb=#' is shown before and after the command, with the command itself appearing on the line between the prompts.

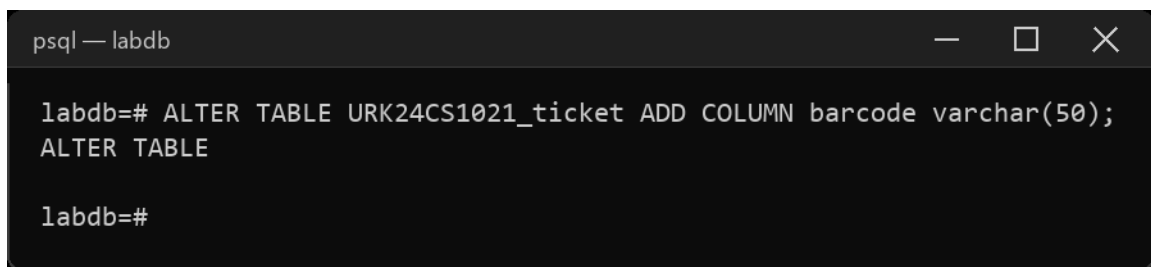
```
psql — labdb

labdb=# ALTER TABLE URK24CS1021_users RENAME COLUMN userid TO id;
ALTER TABLE

labdb=#
```

10. Modify the Ticket table to add a column named "Barcode" with a data type of VARCHAR(50).

```
ALTER TABLE URK24CS1021_ticket ADD COLUMN barcode varchar(50);
```

A terminal window titled 'psql — labdb' with standard window controls. It shows the execution of the SQL command 'ALTER TABLE URK24CS1021_ticket ADD COLUMN barcode varchar(50);'. The prompt 'labdb=#' is shown before and after the command, with the command itself appearing on the line between the prompts.

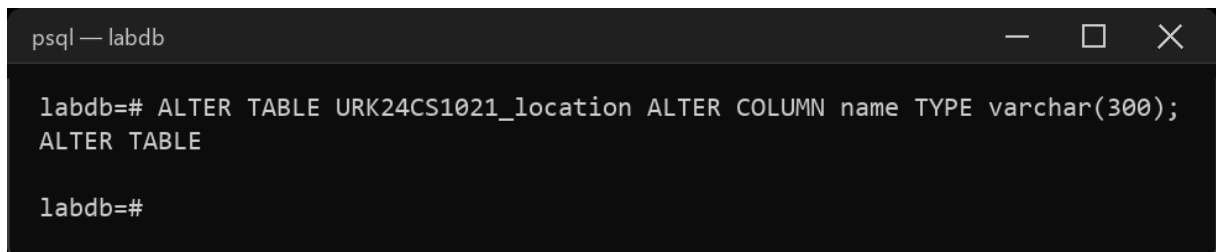
```
psql — labdb

labdb=# ALTER TABLE URK24CS1021_ticket ADD COLUMN barcode varchar(50);
ALTER TABLE

labdb=#
```

11. Modify the Name column in the Venue table to increase its maximum length to VARCHAR(300).

```
ALTER TABLE URK24CS1021_location ALTER COLUMN name TYPE varchar(300);
```

A terminal window titled 'psql — labdb' with standard window controls. It shows the execution of the SQL command 'ALTER TABLE URK24CS1021_location ALTER COLUMN name TYPE varchar(300);'. The prompt 'labdb=#' is shown before and after the command, with the command itself appearing on the line between the prompts.

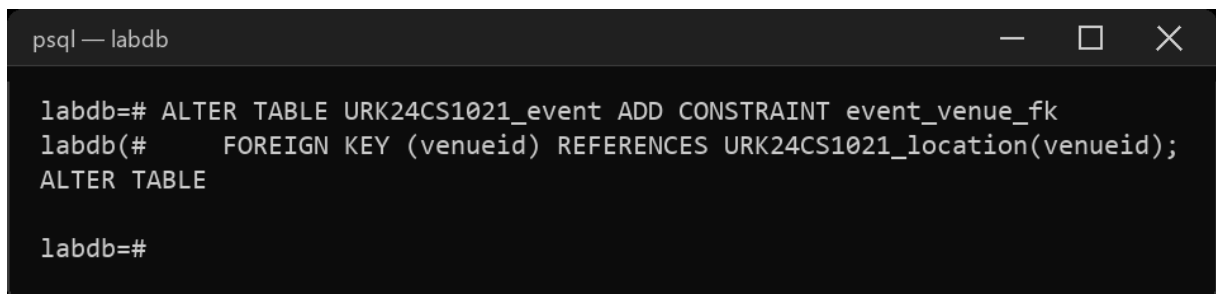
```
psql — labdb

labdb=# ALTER TABLE URK24CS1021_location ALTER COLUMN name TYPE varchar(300);
ALTER TABLE

labdb=#
```

12. Add a foreign key constraint on the VenueID column in the Event table, referencing the Venue table.

```
ALTER TABLE URK24CS1021_event ADD CONSTRAINT event_venue_fk
FOREIGN KEY (venueid) REFERENCES URK24CS1021_location(venueid);
```

A terminal window titled 'psql — labdb' with standard window controls. It shows the execution of the SQL command 'ALTER TABLE URK24CS1021_event ADD CONSTRAINT event_venue_fk FOREIGN KEY (venueid) REFERENCES URK24CS1021_location(venueid);'. The prompt 'labdb=#' is shown before and after the command, with the command itself appearing on the line between the prompts.

```
psql — labdb

labdb=# ALTER TABLE URK24CS1021_event ADD CONSTRAINT event_venue_fk
labdb=# FOREIGN KEY (venueid) REFERENCES URK24CS1021_location(venueid);
ALTER TABLE

labdb=#
```

13. Add a CHECK constraint to check whether the UserID is between 101 and 105.

```
ALTER TABLE URK24CS1021_users ADD CONSTRAINT users_id_chk CHECK (id BETWEEN 101 AND 105);
```

psql — labdb

```
labdb=# ALTER TABLE URK24CS1021_users ADD CONSTRAINT users_id_chk CHECK (id BETWEEN 101 AND 105);  
ALTER TABLE
```

labdb=#

14. Add a unique constraint to Phone column of the User table.

```
ALTER TABLE URK24CS1021_users ADD CONSTRAINT users_phone_uq UNIQUE (phone);
```

psql — labdb

```
labdb=# ALTER TABLE URK24CS1021_users ADD CONSTRAINT users_phone_uq UNIQUE (phone);  
ALTER TABLE
```

labdb=#

15. Truncate the user table.

```
TRUNCATE TABLE URK24CS1021_users;
```

psql — labdb

```
labdb=# TRUNCATE TABLE URK24CS1021_users;  
TRUNCATE TABLE
```

labdb=#

Result

The queries were executed and the desired output was obtained.